

Ejada Egypt Summer Internship Program 2026

Week 2 Technical Documentation

Prepared by: Yara Adel Khedr
github link: [https://github.com/Yara-Khedr/
ejada-intern-terraform](https://github.com/Yara-Khedr/ejada-intern-terraform)

Contents

1	Introduction	3
1.1	Purpose	3
1.2	Environment	3
2	Architecture Diagram	4
3	Network Architecture	5
3.1	VCN and CIDR	5
3.2	Public Subnet	5
3.3	Private Subnet	5
3.4	Gateways	5
3.5	Route Tables	6
4	Security Design	7
4.1	Security Lists vs. NSGs	7
4.2	Public Security List	7
4.3	Private Security List	8
5	Compute, Application Tier and Access Model	9
5.1	YaraVM02	9
5.2	Secure Access Model	9
6	File Storage Service	11
7	Load Balancer	12
8	Terraform Implementation	13
8.1	File Structure	13
8.2	Variables, Locals, Data Sources	13
8.3	Resource Summary	14
8.4	Outputs	14
9	Issues Encountered	16
9.1	Bastion Blocked by IAM	16
9.2	firewalld Blocking Port 80	16
9.3	Cloud-init Mount Hang	16
9.4	Diagnosing Without Bastion or Path Analyzer	16
10	Testing and Validation	17
10.1	Connectivity	17
10.2	Load Balancer	17

10.3 FSS Mount	17
11 Conclusion	18

1. Introduction

1.1 Purpose

This document covers the design, deployment, and validation of the Week 2 network: a two-tier OCI environment with a public Load Balancer and a private application server backed by File Storage, built with Terraform.

1.2 Environment

Attribute		Value
Tenancy		ociejada
Region		me-jeddah-1
Compartment		intern-17-yara-khedr-cmp
Terraform / OCI CLI		1.15.8 / 3.90.1
Terraform	Code	https://github.com/Yara-Khedr/ejada-intern-terraform/tree/main/week2
(Week 2)		

Table 1.1: Environment summary

Resources follow the naming pattern **Yara<Resource>02**, marking them as Week 2 builds.

2. Architecture Diagram

Full network architecture – VCN, subnets, gateways, Load Balancer, compute, and File Storage – including the Bastion.

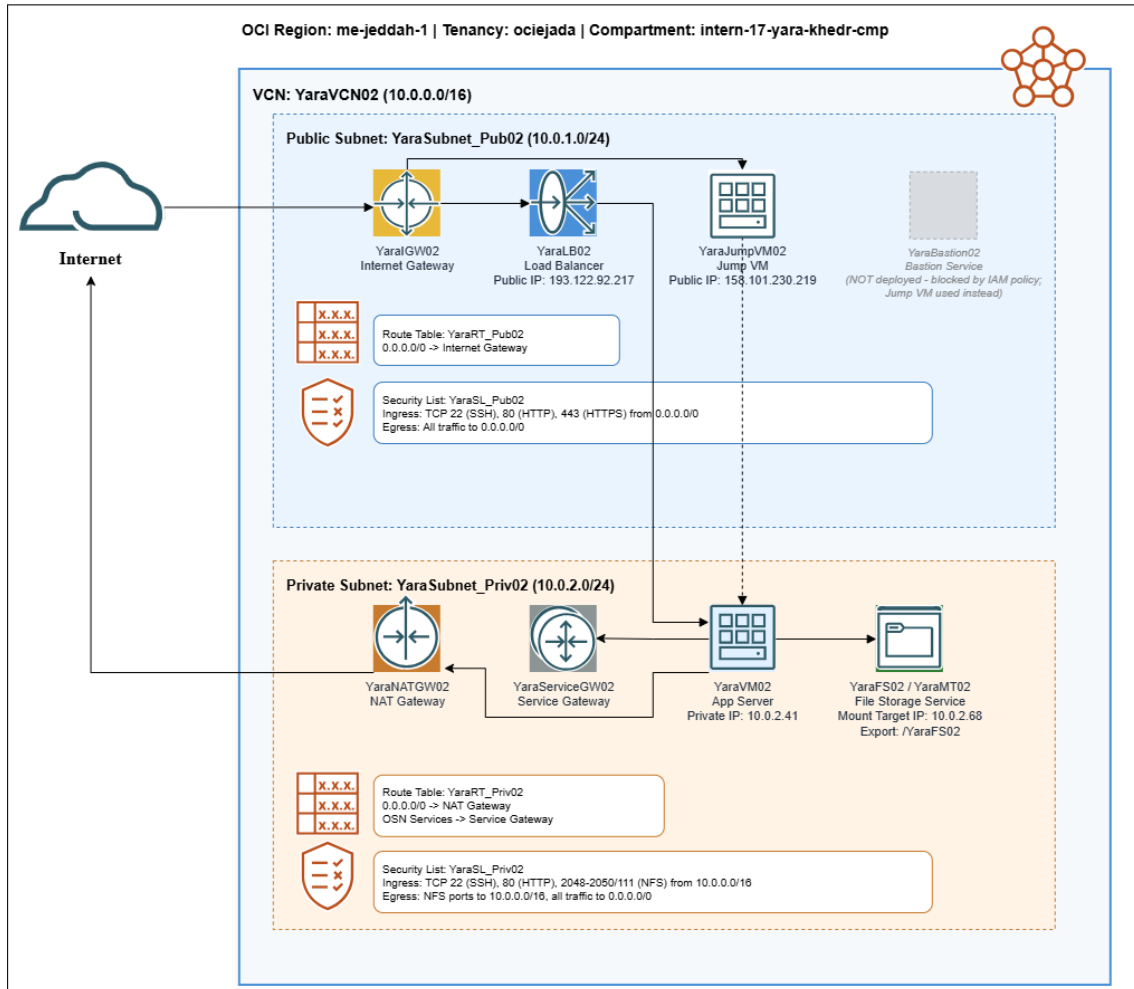


Figure 2.1: Week 2 OCI network architecture

3. Network Architecture

3.1 VCN and CIDR

YaraVCN02 uses 10.0.0.0/16, split into two /24 subnets.

Network	CIDR	Purpose
YaraVCN02	10.0.0.0/16	VCN
YaraSubnet_Pub02	10.0.1.0/24	Public tier
YaraSubnet_Priv02	10.0.2.0/24	Private tier

Table 3.1: CIDR allocation

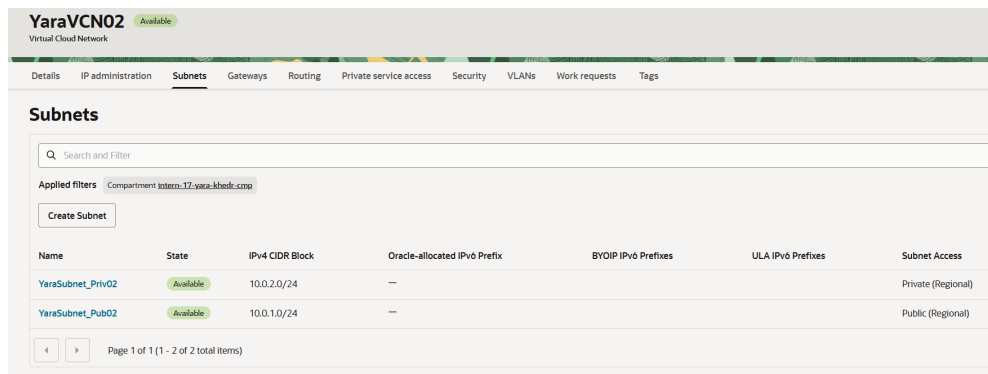


Figure 3.1: VCN YaraVCN02 with both subnets.

3.2 Public Subnet

Hosts **YaraLB02** (Load Balancer, 193.122.92.217) and **YaraJumpVM02** (158.101.230.219). Routed via YaraRT_Pub02 → Internet Gateway.

3.3 Private Subnet

Hosts **YaraVM02** (app server, private only) and **YaraMT02** (FSS mount target). No public IPs. Inbound traffic only via the Load Balancer or Jump VM; outbound via NAT Gateway.

3.4 Gateways

Gateway	Role
YaraIGW02 (Internet GW)	Public subnet in/out internet access
YaraNATGW02 (NAT GW)	Outbound-only internet access for private subnet
YaraServiceGW02 (Service GW)	Private access to OCI services over Oracle backbone

Table 3.2: Gateway roles

3.5 Route Tables

YaraRT_Pub02 Available
Route Table

Details **Route Rules** Tags

Route Rules

Traffic within the VCN is handled by the VCN's local routing by default. Intra-VCN routing allows you more control over routing between subnets. [Learn more](#)

Search and Filter

Add Route Rules Actions

☐	Destination ^	Target Type	Target
☐	0.0.0.0/0	Internet Gateway	YaraIGW02

Page 1 of 1 (1 - 1 of 1 total items)

Figure 3.2: YaraRT_Pub02 – routes to Internet Gateway.

YaraRT_Priv02 Available
Route Table

Details **Route Rules** Tags

Route Rules

Traffic within the VCN is handled by the VCN's local routing by default. Intra-VCN routing allows you more control over routing between subnets. [Learn more](#). If you're having problems

Search and Filter

Add Route Rules Actions

☐	Destination ^	Target Type	Target
☐	0.0.0.0/0	NAT Gateway	YaraNATGW02
☐	All JED Services In Oracle Services Network	Service Gateway	YaraServiceGW02

Page 1 of 1 (1 - 2 of 2 total items)

Figure 3.3: YaraRT_Priv02 – routes to NAT Gateway and Service Gateway.

4. Security Design

4.1 Security Lists vs. NSGs

Security Lists were used instead of NSGs: each subnet has one clear role, so subnet-level rules are sufficient. NSGs suit cases where instances in the *same* subnet need different rules, which doesn't apply here.

4.2 Public Security List

YaraSL_Pub02 Available
Security List

Instance traffic is controlled by firewall rules on each instance in addition to this Security List

Details **Security rules** Tags

Ingress Rules

Search and Filter

Add Ingress Rules Actions

☐	Stateless	Source	IP Protocol	Source Port Range	Destination Port Range	Type and Code	Allows
☐	No	0.0.0.0/0	TCP	All	443		TCP traffic for ports: 443 HTTPS
☐	No	0.0.0.0/0	TCP	All	80		TCP traffic for ports: 80
☐	No	0.0.0.0/0	TCP	All	22		TCP traffic for ports: 22 SSH Remote Login Protocol

Page 1 of 1 (1 - 3 of 3 total items)

Egress Rules

Search and Filter

Add Egress Rules Actions

☐	Stateless	Destination	IP Protocol	Source Port Range	Destination Port Range	Type and Code	Allows
☐	No	0.0.0.0/0	All Protocols				All traffic for all ports

Page 1 of 1 (1 - 1 of 1 total items)

Figure 4.1: YaraSL_Pub02 – ingress 443/80/22 from 0.0.0.0/0; egress all.

4.3 Private Security List

YaraSL_Priv02 Available
Security List

Instance traffic is controlled by firewall rules on each Instance in addition to this Security List

Details **Security rules** Tags

Ingress Rules

Q Search and Filter

Add Ingress Rules Actions

☐	Stateless	Source	IP Protocol	Source Port Range	Destination Port Range	Type and Code	Allows
☐	No	10.0.0.0/16	TCP	All	80		TCP traffic for ports: 80
☐	No	10.0.0.0/16	TCP	All	22		TCP traffic for ports: 22 SSH Remote Login Protocol

Page 1 of 1 (1 - 2 of 2 total items)

Egress Rules

Q Search and Filter

Add Egress Rules Actions

☐	Stateless	Destination	IP Protocol	Source Port Range	Destination Port Range	Type and Code	Allows
☐	No	0.0.0.0/0	All Protocols				All traffic for all ports
☐	No	10.0.0.0/16	UDP	All	111		UDP traffic for ports: 111
☐	No	10.0.0.0/16	TCP	All	2048-2050		TCP traffic for ports: 2048-2050
☐	No	10.0.0.0/16	TCP	All	111		TCP traffic for ports: 111
☐	No	10.0.0.0/16	UDP	All	2048		UDP traffic for ports: 2048

Page 1 of 1 (1 - 5 of 5 total items)

Figure 4.2: YaraSL_Priv02 – ingress restricted to 10.0.0.0/16; egress includes NFS ports.

Design Note

Private-subnet ingress is limited to 10.0.0.0/16 so YaraVM02 only accepts traffic originating inside the VCN – never directly from the internet.

5. Compute, Application Tier and Access Model

5.1 YaraVM02

Oracle Linux 9, VM.Standard.A1.Flex (1 OCPU, 6 GB), private subnet, no public IP.

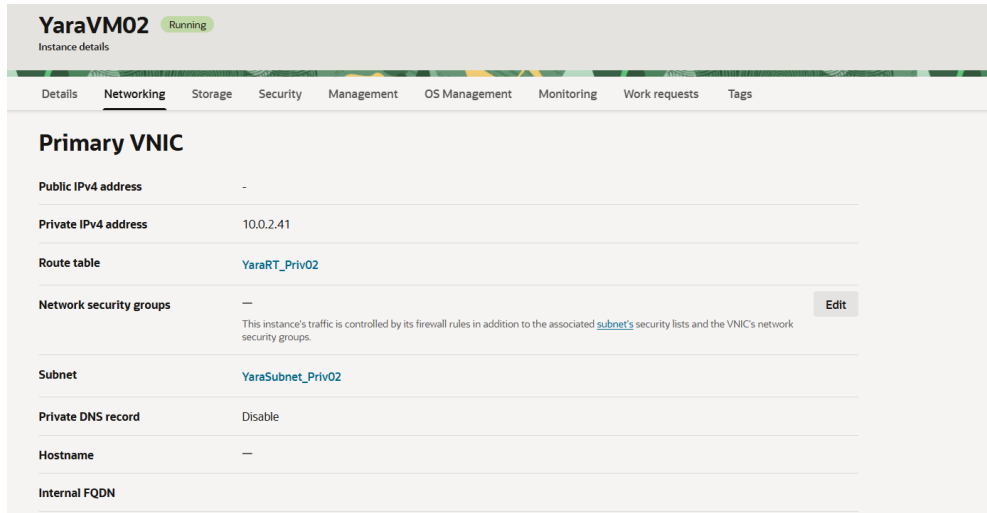


Figure 5.1: YaraVM02 – private IP 10.0.2.41.

Cloud-init handles setup on boot: installs `nfs-utils/httpd`, mounts FSS at `/mnt/appdata` (persisted via `/etc/fstab`), writes `index.html`, opens port 80 in `firewalld`, and enables `httpd`.

5.2 Secure Access Model

Design intent: OCI Bastion Service (session-based, no permanent host).

Blocker: YaraBastion02 failed – IAM policy in the compartment prevented Bastion provisioning.

Resolution: YaraJumpVM02 deployed in the public subnet as the SSH entry point, restricted by YaraSL_Pub02/YaraSL_Priv02 to keep the same network-level guarantee – no direct path from the internet to YaraVM02.

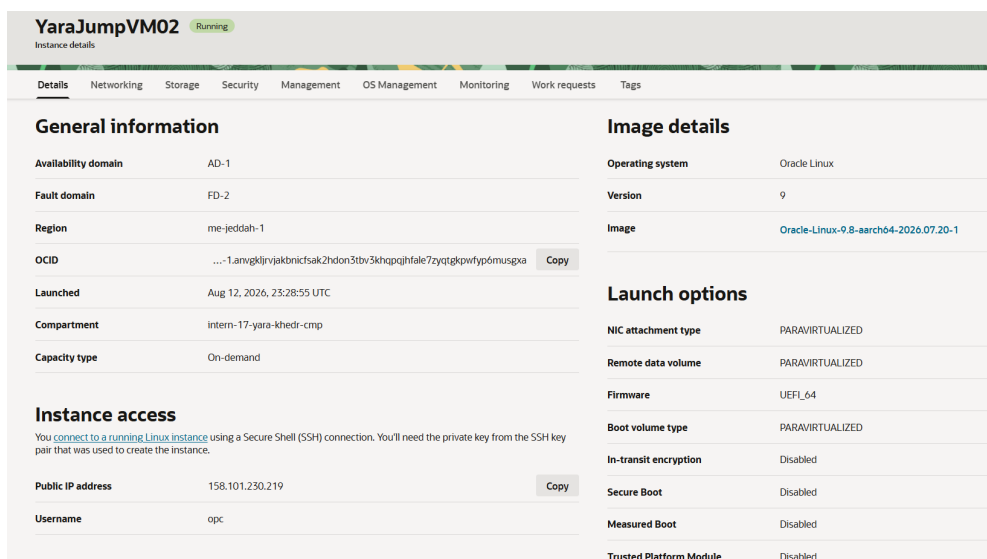


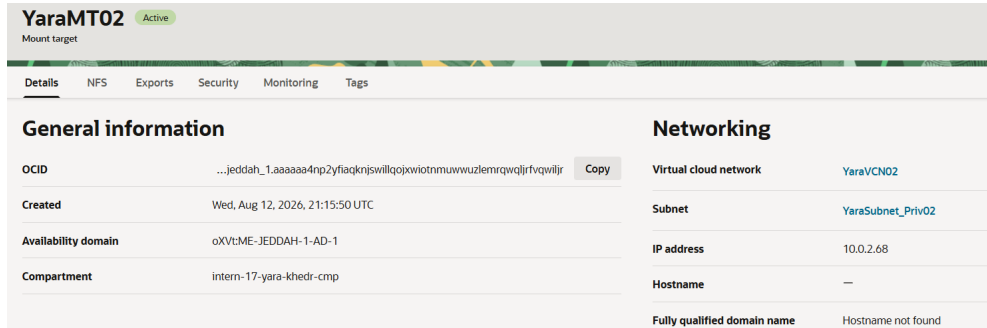
Figure 5.2: YaraJumpVM02 – public IP 158.101.230.219.

Method	Status
Cloud Shell	Available for quick checks
Jump VM	Used – Terraform-managed SSH relay
Bastion Service	Designed, blocked by IAM
SSH Tunneling	Used through the Jump VM

Table 5.1: Private-VM access options

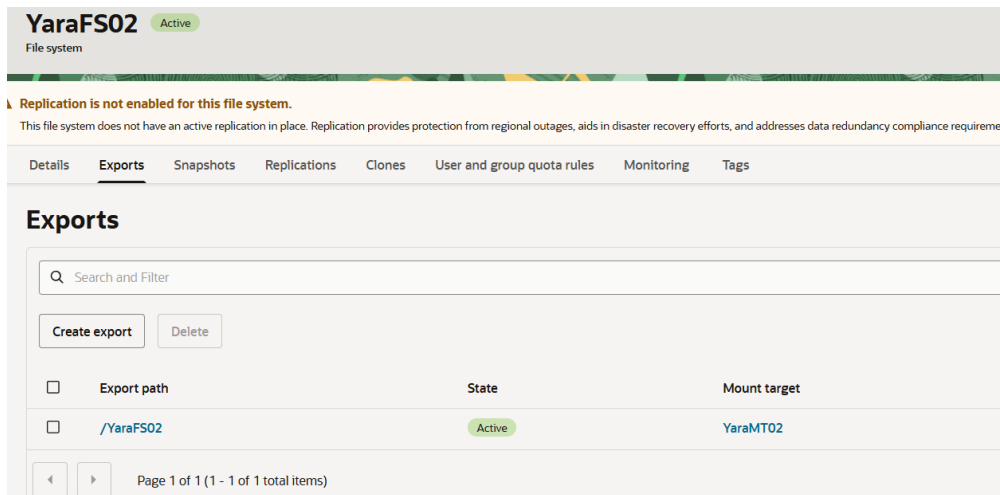
6. File Storage Service

YaraFS02 (file system) is exposed via mount target YaraMT02 (10.0.2.68) through export /YaraFS02.



YaraMT02 Active	
Mount target	
Details NFS Exports Security Monitoring Tags	
General information	
OCID	...jeddah_1.aaaaa4np2yfiagknjswillqojxwiotnmuwuzlemrqwjlrfrvqwljr Copy
Created	Wed, Aug 12, 2026, 21:15:50 UTC
Availability domain	oXVtME-JEDDAH-1-AD-1
Compartment	intern-17-yara-khedr-cmp
Networking	
Virtual cloud network	YaraVCN02
Subnet	YaraSubnet_Priv02
IP address	10.0.2.68
Hostname	—
Fully qualified domain name	Hostname not found

Figure 6.1: YaraMT02 mount target – private IP 10.0.2.68.



YaraFS02 Active		
File system		
Replication is not enabled for this file system. This file system does not have an active replication in place. Replication provides protection from regional outages, aids in disaster recovery efforts, and addresses data redundancy compliance requirements.		
Details Exports Snapshots Replications Clones User and group quota rules Monitoring Tags		
Exports		
Search and Filter		
Create export Delete		
☐	Export path	State
☐	/YaraFS02	Active
		Mount target
		YaraMT02
Page 1 of 1 (1 - 1 of 1 total items)		

Figure 6.2: YaraFS02 export – path /YaraFS02.

Mounted automatically by cloud-init at /mnt/appdata; NFS ports (111, 2048–2050) are opened between the private subnet and mount target via YaraSL_Priv02.

7. Load Balancer

YaraLB02: Flexible shape, 10–100 Mbps, public IP 193.122.92.217.

Listener YaraListener02 (port 80) forwards to backend set YaraBES02 (ROUND_ROBIN), backend = YaraVM02:80.

The screenshot displays the Azure portal interface for the load balancer YaraLB02. The top navigation bar includes tabs for Details, Security, Listeners, Backend sets, Policies, Certificates and ciphers, Hostnames, Monitoring, Work requests, and Tags. The main content area is divided into two sections: 'Load balancer health' and 'Load balancer information'.

Load balancer health

Overall health	
Overall health	OK

Backend sets health

Health	Count
Critical	0
Warning	0
Incomplete	0
Pending	0
OK	1

Backend sets drain status

Status	Count
Drained	0

Load balancer information

Traffic between this load balancer and its backend servers is subject to the governing security lists and network security groups.

[Learn more about load balancers and security lists](#)

OCID	...-1.aaaaaaaan9l2scfwigf5go6wgrcbzbawpctxz4x42mdndhy754lurbtq	Copy
Created	Aug 12, 2026, 21:55:28 UTC	
Shape	Flexible	
Min bandwidth	10 Mbps	
Max bandwidth	100 Mbps	
IP address	193.122.92.217 (public)	
Virtual cloud network	YaraVNet02	
Subnet	YaraSubnet_Pub02	

Figure 7.1: YaraLB02 – health OK, one backend healthy.

Browsing the public IP returns the page served by YaraVM02, confirming the full path works end-to-end.

8. Terraform Implementation

8.1 File Structure

Code lives in `week2/` (Week 1 untouched in `week1/`): <https://github.com/Yara-Khedr/ejada-intern-terraform/tree/main/week2>

File	Purpose
<code>providers.tf</code>	OCI provider config
<code>variables.tf</code> / <code>terraform.tfvars</code>	Inputs
<code>data.tf</code>	Data source lookups
<code>locals.tf</code>	Derived values
<code>main.tf</code>	Resources
<code>outputs.tf</code>	Outputs
<code>cloud-init.sh</code>	Boot-time app setup (templated)

Table 8.1: File structure – week2/

8.2 Variables, Locals, Data Sources

Values that vary by environment – compartment ID, subnet CIDRs, instance sizing, Load Balancer configuration, and the application port – are pulled from variables. Locals are used for values derived from data sources: the Service Gateway’s service ID and CIDR, and the availability domain name. Display names and protocol-level constants (TCP/UDP numbers, NFS ports) remain as literals, since they define the naming convention and protocol behavior rather than environment-specific configuration.

Variable	Purpose
<code>region</code> , <code>compartment_id</code>	Target region / compartment
<code>vcn_cidr</code> , <code>allowed_ingress_cidr</code>	VCN sizing and public ingress scope
<code>public_subnet_cidr</code> , <code>private_subnet_cidr</code>	Subnet sizing per tier
<code>instance_shape/ocpus/memory_gbs</code>	App server compute sizing
<code>jump_vm_ocpus</code> , <code>jump_vm_memory_gbs</code>	Jump VM compute sizing
<code>ssh_public_key_path</code>	Instance SSH access
<code>fss_export_path</code>	NFS export path for File Storage
<code>lb_shape/min_bandwidth/max_bandwidth</code>	Load Balancer shape and bandwidth range
<code>app_port</code>	Application port (backend, health check, listener)
<code>bastion_allowed_cidr</code>	Reserved for Bastion (unused – IAM blocked)

Table 8.2: Key variables

Local	Purpose
<code>all_services_id</code>	Oracle Services Network OCID for Service GW
<code>all_Services_cidr</code>	Associated CIDR label for route rules
<code>availability_domain_name</code>	AD used for compute placement

Table 8.3: Key locals

8.3 Resource Summary

Type	Name	Purpose
VCN	YaraVCN02	10.0.0.0/16
Subnet	YaraSubnet_Pub02	Public, 10.0.1.0/24
Subnet	YaraSubnet_Priv02	Private, 10.0.2.0/24
Internet GW	YaraIGW02	Public internet access
NAT GW	YaraNATGW02	Outbound-only for private tier
Service GW	YaraServiceGW02	Private access to OCI services
Route Table	YaraRT_Pub02	Public routing
Route Table	YaraRT_Priv02	Private routing
Security List	YaraSL_Pub02	Public firewall rules
Security List	YaraSL_Priv02	Private firewall rules
Compute	YaraVM02	App server (private)
Compute	YaraJumpVM02	Jump host (public)
File System	YaraFS02	App data storage
Mount Target	YaraMT02	NFS access point
Export	YaraExport02	/YaraFS02
Load Balancer	YaraLB02	Public entry point
Backend Set	YaraBES02	ROUND_ROBIN
Listener	YaraListener02	HTTP :80
Bastion	YaraBastion02	Written, commented out (IAM blocked)

Table 8.4: Resource inventory

8.4 Outputs

Output	Value
<code>lb_public_ip</code>	193.122.92.217
<code>vm_private_ip</code>	10.0.2.41
<code>jump_vm_public_ip</code>	158.101.230.219
<code>mount_target_ip</code>	10.0.2.68
<code>fss_export_path</code>	/YaraFS02

Table 8.5: Terraform outputs

```
Apply complete! Resources: 2 added, 0 changed, 2 destroyed.  
  
Outputs:  
  
fss_export_path = "/YaraFS02"  
jump_vm_public_ip = "158.101.230.219"  
lb_public_ip = "193.122.92.217"  
mount_target_ip = "10.0.2.68"  
vm_private_ip = "10.0.2.41"
```

Figure 8.1: Final terraform apply – success, all outputs.

9. Issues Encountered

9.1 Bastion Blocked by IAM

Cause: IAM policy in the compartment prevented Bastion provisioning.

Fix: Jump VM deployed as the working alternative.

9.2 firewalld Blocking Port 80

Cause: Oracle Linux 9 ships with `firewalld` active, blocking inbound port 80 by default – independent of OCI Security List rules.

```
firewall-cmd --permanent --add-service=http
firewall-cmd --reload
```

```
[opc@YaraVM02 ~]$ sudo firewall-cmd --permanent --add-service=http
sudo firewall-cmd --reload
sudo firewall-cmd --list-all
success
success
public (active)
  target: default
  icmp-block-inversion: no
  interfaces: enp0s6
  sources:
  services: dhcpv6-client http ssh
  ports:
  protocols:
  forward: yes
  masquerade: no
  forward-ports:
  source-ports:
  icmp-blocks:
  rich rules:
[opc@YaraVM02 ~]$ |
```

Figure 9.1: firewalld fix applied on YaraVM02.

Lesson

Network-layer rules (Security Lists) and host firewalls are independent – both must allow the traffic.

9.3 Cloud-init Mount Hang

Cause: Instance placed in the wrong subnet has no route/permission to the FSS mount target, so the NFS mount hangs at boot.

Fix: confirmed YaraVM02 launches into YaraSubnet_Priv02 only.

9.4 Diagnosing Without Bastion or Path Analyzer

Console history only showed early boot output, not cloud-init logs, and Network Path Analyzer was blocked by the same IAM restriction as the Bastion.

Fix: diagnosed manually through the Jump VM – reading `/var/log/cloud-init-output.log` directly via SSH, and running `tcpdump` to confirm traffic was reaching YaraVM02, which isolated the port-80 issue to the guest firewall.

10. Testing and Validation

10.1 Connectivity

SSH to YaraVM02 verified via Jump VM; unreachable directly from the internet, as designed.

10.2 Load Balancer

Backend health OK: browser request to the public IP returns the app page.



Figure 10.1: End-to-end verification via <http://193.122.92.217>.

10.3 FSS Mount

Export confirmed mounted and writable at `/mnt/appdata`; `/etc/fstab` entry persists the mount across reboots.

11. Conclusion

Week 2 delivered a two-tier OCI network, fully defined in Terraform: a public Load Balancer, a private application server, and File Storage, all verified working end-to-end.

Beyond the deployment itself, this week reinforced a few broader principles that carry into any cloud environment: security must be enforced at multiple layers, not just one; real environments come with constraints (IAM, permissions, service limits) that require practical workarounds rather than blocking progress; and when high-level tools aren't available, first-principles troubleshooting still gets the job done.